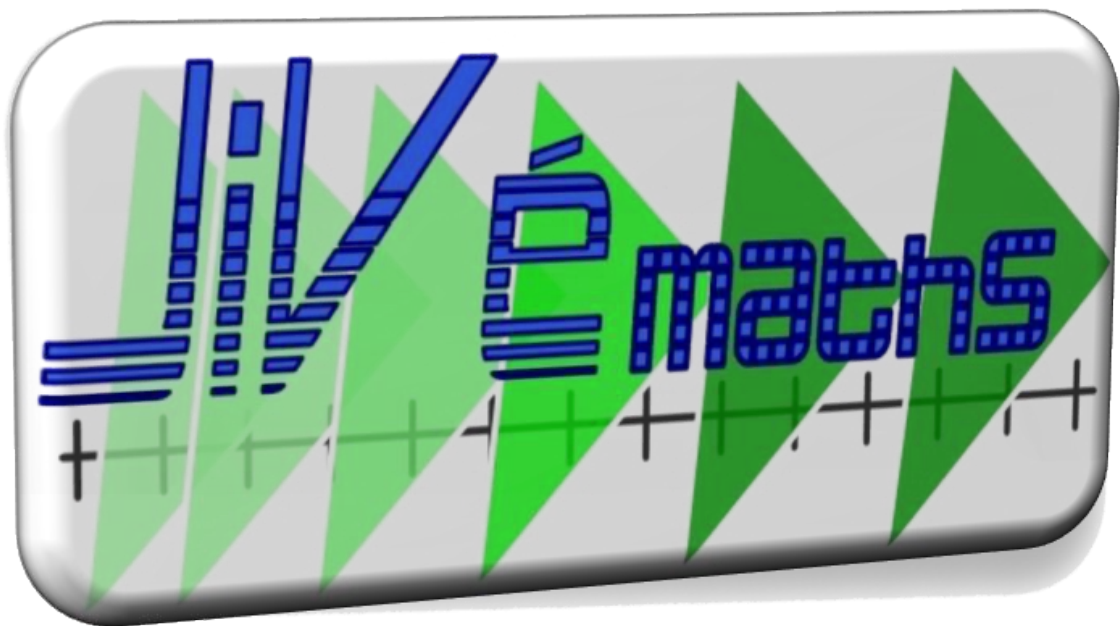


« JiVé Maths »

Etude et développement de solutions 3D pour la réalisation d'un prototype de jeu éducatif basé sur les mathématiques



Jeux **I**nteractifs **V**irtuels pour l'**E**ducation

Sommaire

Introduction	1
→ Contexte	1
→ Finalité du projet	1
Travail préparatoire	2
→ Les besoins de l'entreprise	2
→ Organisation du projet	2
→ Etude de solutions 3D	3
Base de données	5
→ Choix des outils	5
→ Conception et génération	5
→ Mapping objet-relationnel	7
Prototype 3D	8
→ Choix des outils	8
→ Diagramme de classes	8
→ Les éléments du monde virtuel	10
→ Déplacement dans un monde virtuel	11
→ Pointer et manipuler des éléments 3D	13
→ Activité du prototype : « La balance »	14
Conclusion	16
→ Etat actuel du projet	16
→ Bilan personnel	16

Introduction

Contexte

Le CERV (Centre Européen de Réalité Virtuelle) est un centre de recherche affilié à l'ENIB (Ecole Nationale d'Ingénieurs de Brest). Créé en 2004, il est constitué de trois grands pôles : recherche, enseignement et valorisation des travaux.

Ses domaines d'activités sont nombreux et couvrent l'ensemble des branches de la réalité virtuelle. Un exemple de réalisation des entreprises affiliées au CERV est la modélisation de la circulation à Brest une fois le tramway installé.



Le service valorisation aide les entreprises localisées au CERV à se développer pour peu qu'elles utilisent le fruit de la recherche qui y est faite, et contribuent à l'améliorer.

C'est dans ce contexte que se déroule mon projet, au sein de l'entreprise en cours de création JiVé Maths, sous la responsabilité de l'un de ses co-fondateurs, aussi doctorant en informatique et enseignant à l'ENIB.

Finalité du projet

Le projet JiVé Maths porté par l'entreprise éponyme consiste en un jeu vidéo massivement multijoueur ayant pour but l'apprentissage des mathématiques.

La première version de ce logiciel concernera les collégiens de niveau quatrième et sera par la suite adaptée aux classes de niveau plus élevé, pour arriver progressivement à la couverture totale des formations enseignant les mathématiques.

Basé sur le référentiel de l'éducation nationale, le jeu vidéo présentera au joueur une suite de problèmes solubles grâce aux mathématiques. Leur résolution permettra un suivi et une validation des compétences de l'élève.

A terme, ce jeu sérieux (de l'anglais « serious game ») aura pour vocation de servir de support de cours pour les enseignants.

A mon arrivée dans les locaux de l'entreprise, le logiciel n'est encore qu'à l'état de projet. Je dispose toutefois de modèles 3D utilisables et de scénarios d'activités établis.

Travail préparatoire

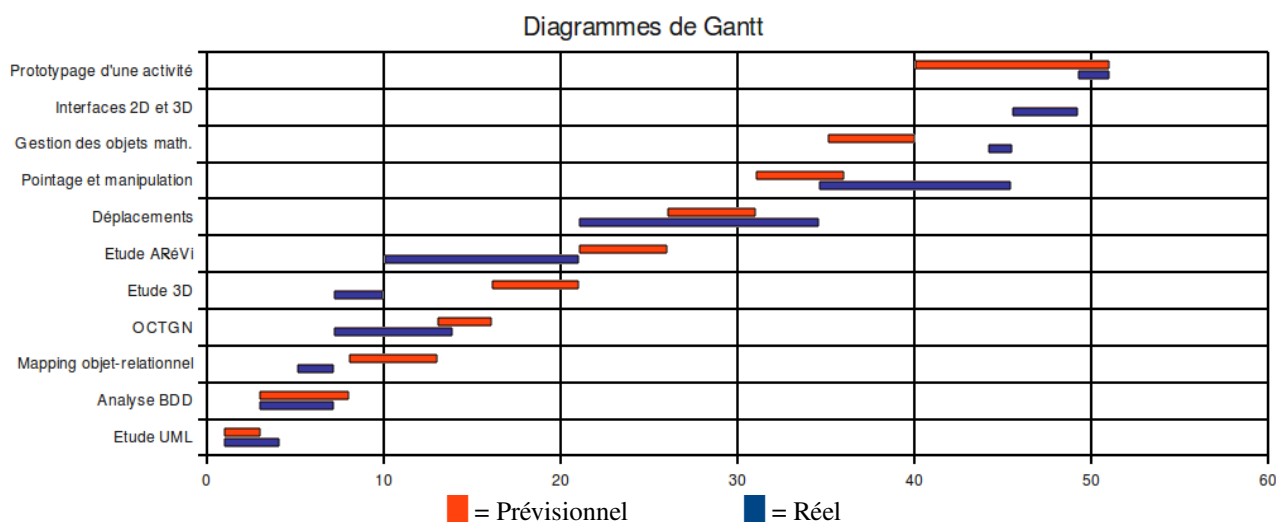
Les besoins de l'entreprise

Mon travail au cours de cette période consiste à fournir un prototype d'activité présentable lors de conférences, permettant ainsi l'appui de dossiers de demande de crédits.

Le logiciel devra prendre en compte un certain nombre de contraintes :

- Il devra être multi-plateformes pour permettre un plus vaste accès.
- Les déplacements dans le monde virtuel devront permettre une navigabilité intuitive.
- Les activités ludiques devront être proposées aux joueurs en fonction de leur niveau.
- Le prototype devra présenter une activité illustrant la philosophie de JiVé Maths.
- Le suivi des capacités de l'élève par ses parents pourra se faire via un portail Web.
- Le code généré devra être réutilisable dans le modèle d'application final.

Organisation du projet



On peut remarquer un certain nombre de différences entre les diagrammes. En effet, la partie base de données a été plus rapide que prévu, probablement grâce au fait que j'en avais déjà réalisées. Au contraire, les développements concernant la 3D m'ont pris bien plus de temps car je n'avais pas réalisé lors de la planification à quel point la bibliothèque *C++ ARéVi* était grande. Je me suis donc retrouvé avec moins de temps que je l'avais espéré pour programmer l'activité du prototype.

Etude de solutions 3D



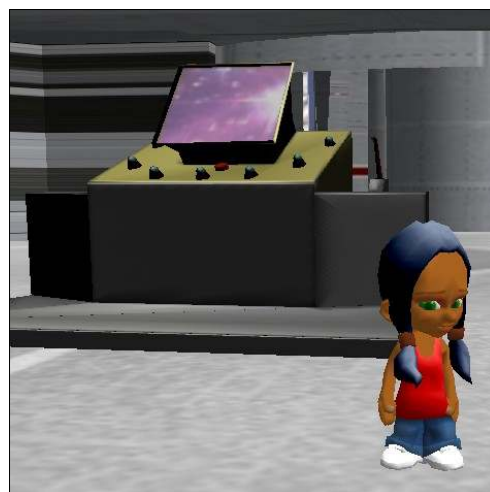
Solipsis & Artefacto

Solipsis est un moteur de jeu libre intégrant *Ogre* (3D) et *Raknet* (réseau). Basé sur le peer-to-peer, il offre la possibilité d'héberger son propre monde mais aussi d'accéder à des mondes distants, ce qui correspond à nos besoins.

L'entreprise Artefacto, que nous avons contactée pour un devis de sous-traitance de l'aspect 3D du logiciel (capture ci-contre), nous a fourni quelques captures d'écran provenant d'un monde virtuel de leur conception utilisant *Solipsis* (capture ci-dessous).

Toutefois, si le rendu semble correspondre aux attentes du projet, la modélisation de cinq activités mathématiques nécessite auprès de cette entreprise un budget trop important pour choisir cette solution.

Nous avons alors tenté d'utiliser directement le moteur de jeu *Solipsis* mais nous avons vite abandonné cette option car il s'est avéré être très difficilement portable sur une autre plateforme que *Windows*. De plus, le projet n'est plus maintenu depuis 2008, et contient encore un grand nombre de bugs dont certains très importants.



Python & Pyglet

N'ayant pas trouvé de solution satisfaisante parmi les modèles préconçus de mondes 3D, nous avons décidé de programmer le nôtre.

La première solution qui m'est apparue a été l'utilisation de la bibliothèque *Pyglet* et du langage *Python*, gratuits et multi-OS. Des captures de logiciels trouvées sur le web m'ont permis de constater que ces outils pouvaient nous permettre de réaliser les besoins du projet.

Toutefois, si ce langage offre une impressionnante facilité d'apprentissage, ce n'est hélas pas le cas de *Pyglet* qui n'est qu'une simple intégration d'*OpenGL* (interface de programmation 3D) pour *Python*. Après quelques essais, je me suis vite rendu compte que je n'avais pas assez de connaissances en matière de programmation bas niveau, et j'ai décidé de laisser de côté cette bibliothèque car je n'avais que peu de temps en entreprise pour m'y former.

Python & Panda3D

Panda3D est un moteur de jeu complet pour *Python* développé par les studios Disney Interactive. Bien plus simple d'utilisation que *Pyglet*, *Panda3D* offre de plus une documentation très riche ainsi que de nombreux exemples.

Mais malgré cela, il a fallu trouver une autre solution car ce moteur ne fonctionnait pas de la même manière d'un OS à l'autre. En effet, le rendu était impeccable sous *Windows 7*, mais de nombreux éléments refusaient de s'afficher sous *Ubuntu 9.10*.



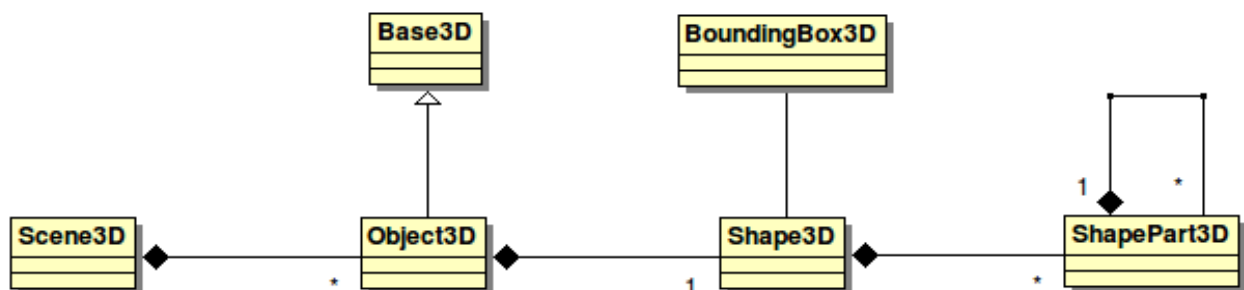
ARéVi – Atelier de Réalité Virtuelle

ARéVi est une bibliothèque du langage *C++* disposant de nombreux avantages. Développée et maintenue au CERV, elle offre notamment les services suivants :

- Simplification du *C++* grâce à un mécanisme de macros et de templates.
- Des pointeurs intelligents, pour s'affranchir de la gestion de mémoire propre au *C++*.
- Nombreuses classes et fonctions permettant une manipulation simplifiée de la 3D.
- Documentation et soutien, ainsi qu'un tutoriel d'apprentissage.

Mon choix s'est donc porté sur cette option car *ARéVi* permet de répondre aux contraintes définies pour la réalisation du prototype. En effet, cette bibliothèque est multi-plateformes et permettra une facile réutilisation du code généré, du fait de sa localisation au CERV.

La gestion de la 3D en *ARéVi* se fait comme suit :



Prenons le cas d'un cube (*Shape3D*). Il est constitué de faces (*ShapePart3D*) et peut être englobé par une boîte invisible (*BoundingBox3D*). On décide de faire de ce cube un objet manipulable (*Object3D*). Il pourra alors obtenir des propriétés et méthodes communes à ce qui est en 3D (*Base3D*), et apparaître à l'écran dans des scènes (*Scene3D*) affichant d'autres objets 3D.

Base de données

Choix des outils

Afin de modéliser la base de données qui servira au suivi des joueurs, j'ai utilisé le logiciel *BOUML*. C'est un outil d'analyse *UML* souple, permettant la génération automatique de classes *Python*, *C++*, *PHP* et *Java* à partir des informations des modèles.

Quant au à l'hébergement de cette base de données, j'ai opté pour un serveur *PostgreSQL*, qui a été installé sur un OS *Ubuntu 9.10*.

Conception et génération

Le MLD suivant permet de mieux comprendre le diagramme ayant servi à la conception de la base de données de gestion des utilisateurs et des activités, figurant en page 6.

activity_composed_activity	(<i>composedActivity</i> , <i>composingActivity</i>);
activity_concerns_skill	(<i>activity</i> , <i>skill</i> , type, importance);
activitysequence_groups_activity	(<i>activitySequence</i> , <i>activity</i>);
knowledge_illustrates_ability	(<i>knowledge</i> , <i>ability</i>);
player_learns_skill	(<i>player</i> , <i>skill</i> , progress);
player_uses_activity	(<i>player</i> , <i>activity</i> , progress);
subchapter_contains_skill	(<i>program</i> , <i>chapter</i> , <i>subchapter</i> , <i>skill</i>);
ability	(<i>identifier</i>);
activity	(<i>identifier</i> , <i>teacher</i> , description);
activitysequence	(<i>identifier</i> , <i>teacher</i> , description);
chapter	(<i>program</i> , <i>identifier</i> , description);
customer	(<i>login</i>);
knowledge	(<i>identifier</i>);
person	(<i>login</i> , password, name, firstName, mail, specialization);
player	(<i>login</i> , <i>customer</i>);
program	(<i>year</i>);
skill	(<i>identifier</i> , <i>theme</i> , description, specialization);
subchapter	(<i>program</i> , <i>chapter</i> , <i>identifier</i> , description);
teacher	(<i>login</i>);
theme	(<i>identifier</i> , description);

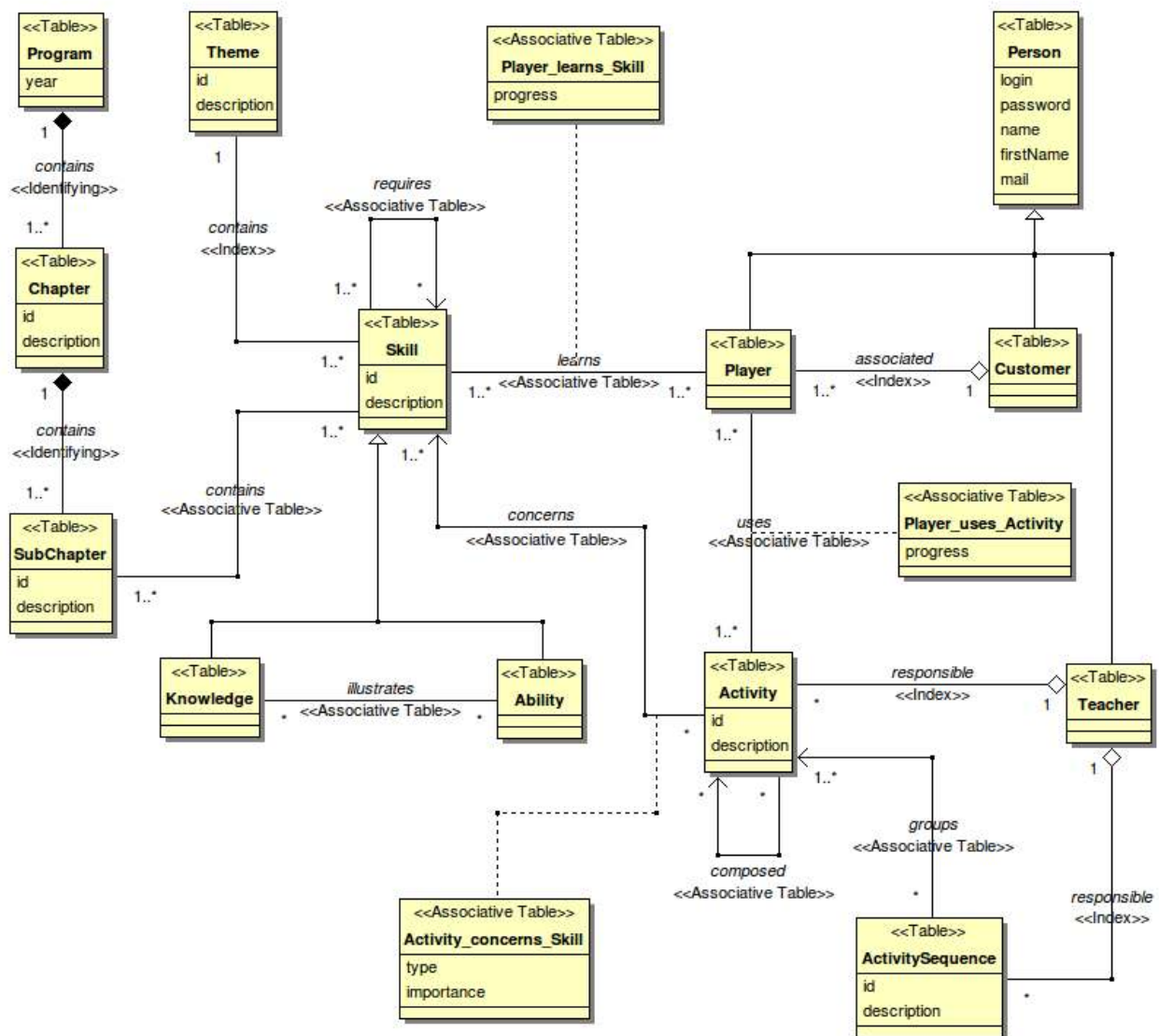
Table

Clé primaire

Clé étrangère

Le langage de modélisation *UML* a été utilisé plutôt que *Merise* afin de faciliter le mapping objet-relationnel. En effet, l'utilisation de stéréotypes *UML* a permis de modéliser d'une part les

tables constituant la base de données, mais aussi les classes associées en *Python*.



La traduction littérale des principaux axes de ce diagramme peut se faire ainsi :

- Le programme d'une année se compose de chapitres, eux-mêmes constitués de sous-chapitres qui ont pour but de développer des compétences pratiques et théoriques chez l'élève.
- L'étude d'une compétence nécessite un certain nombre de pré-requis. Par exemple, un élève étudiant le théorème de Pythagore devra d'abord savoir ce qu'est un triangle rectangle.
- Les activités mettent en œuvre des compétences, et sont gérées par des enseignants. Des élèves y participent, permettant donc de connaître leur niveau pour la compétence illustrée.
- Chaque personne dispose d'un profil lui donnant des avantages selon son statut.

Cette analyse permet bien un suivi des compétences de l'élève, mais met aussi en place les jalons pour un système d'authentification sur le portail communautaire qui sera développé afin de permettre aux parents et aux enseignants de consulter l'avancement des élèves au sein des différentes

activités.

De plus, elle permet la prise en compte du programme officiel de l'éducation nationale sur lequel seront basées les activités du logiciel JiVé Maths.

L'implémentation de cette base de données sur le serveur *PostgreSQL* a été réalisée depuis un poste distant grâce à un script *Python* de ma confection.

..... Mapping objet-relationnel

Le mapping objet-relationnel est une technique permettant de recréer des objets représentant une base de données. Il facilite la manipulation des informations et aide à garder de manière simple une base de données à jour au fil de l'exécution du programme.

Pour permettre cela, j'ai étudié le langage *Python*, et plus particulièrement sa bibliothèque *SQLAlchemy*. Elle permet une simplification des requêtes *SQL* grâce à des fonctions et à un mécanisme de session : la base de données est téléchargée lors de la création des classes et est stockée en local. Des liens sont créés afin de signaler à *SQLAlchemy* que les champs des tables récupérées sont associés aux propriétés de ces classes. Toute modification des objets instanciés à partir de ces classes est automatiquement reportée dans cette copie de la base de données.

Cette technique permettra au chargement du prototype de récupérer le profil de l'utilisateur. L'évolution dans le monde virtuel se fera donc sans interactions majeures avec la base de données, ce qui évitera des ralentissements.

De manière ponctuelle, il sera possible de sauvegarder l'évolution du joueur en synchronisant la session et la base de données distante. Ce mapping objet-relationnel permettra aussi de garder en mémoire la position actuelle de l'élève dans le jeu en cas de coupure soudaine de la connexion Internet.



Prototype 3D

Choix des outils

Afin de générer le code du prototype, j'ai en premier lieu utilisé un éditeur de texte offrant les habituels services d'indentation, de coloration syntaxique et d'auto-complétion : *Geany*. Après quelques jours, j'ai migré vers le même logiciel qui m'avait permis de réaliser la base de données : *BOUML*. En effet, en complément de ses outils *UML* pour manipuler les classes, ce dernier autorise la saisie du code des méthodes, permettant de regrouper analyse et programmation.

Le code écrit, j'ai utilisé *SCons* et *gcc* comme outils de compilation pour générer mes fichiers exécutables, et *Valgrind* comme outil d'analyse des erreurs.

Les éléments 3D ont été créés avec *Google SketchUp*, logiciel de modélisation offrant une version gratuite permettant l'export au format standard *Collada*, utilisable dans *ARéVi*.

Diagramme de classes

L'analyse des besoins pour la programmation du prototype a pour but de lier les classes concernant le projet JiVé Maths entre elles, et de les greffer sur celles appartenant à la bibliothèque *ARéVi*, afin de profiter de ses avantages.

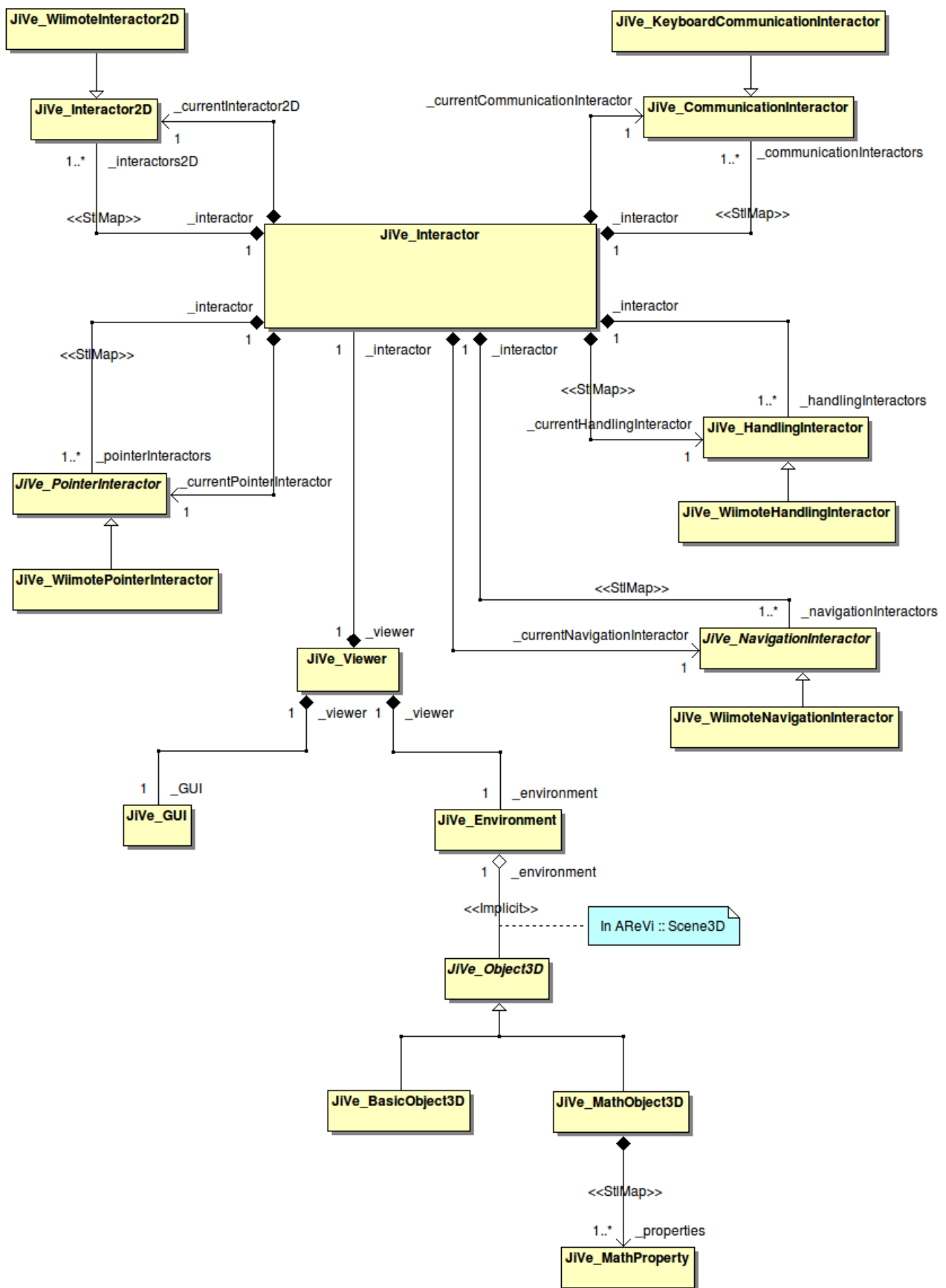
Cette analyse a nécessité plusieurs diagrammes de classes. L'un d'entre eux, représentant l'ensemble des interactions entre les classes du projet, est disponible en page 9.

On peut distinguer sur le diagramme trois catégories de classes correspondant au MVC (modèle – vue – contrôleur) : les classes concernant la gestion des objets, celles s'occupant de l'affichage dans le jeu et celles définissant les interacteurs :

→ M : La séparation des objets en deux catégories, basiques et mathématiques, a pour but de déterminer quels objets seront manipulables au cours des activités. Les propriétés associées permettront par exemple d'attribuer un poids à une pomme ou une hauteur à un mur.

→ V : L'environnement est l'endroit où seront insérés les objets. Il dispose de scènes dans lesquelles les disposer, et envoie un rendu sur la fenêtre de jeu (*JiVe_Viewer*).

→ C : Les cinq catégories d'interacteurs permettent la gestion du monde virtuel. Ainsi, l'interacteur de navigation servira par exemple à faire avancer ou voler le personnage, alors que celui de pointage placera le curseur au bon endroit de l'écran et pourra déterminer quel objet 3D est visé. Tous sont gérés dans la classe *JiVe_Interactor* afin de garantir une cohésion entre eux.



..... Les éléments du monde virtuel

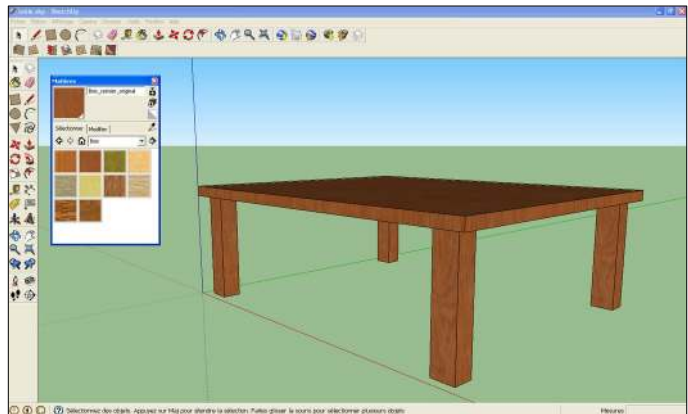
Avant toute chose, il convient de clarifier les termes utilisés. Nous parlerons donc d'éléments afin de désigner les choses apparaissant dans le monde virtuel (maisons, tables...). Ce terme sera préféré à celui d'objets, qui se rapportera aux instances de classes.

Cette section a pour but de suivre un élément 3D de sa création à son utilisation au sein du logiciel. Nous prendrons donc l'exemple d'une table.

Conception

La première étape dans le processus d'utilisation d'un élément est sa conception. Pour ce faire, j'ai réalisé divers modèles sous *Google SketchUp*.

Une fois terminés, ils sont exportés au format standard *Collada*, que nous allons pouvoir exploiter au sein du logiciel.



Caractérisation

La seconde étape est ce qu'on peut appeler la caractérisation de l'élément. C'est sa déclaration dans le monde virtuel comme objet, et de manière générale tout de qui peut caractériser la table au sein du logiciel.

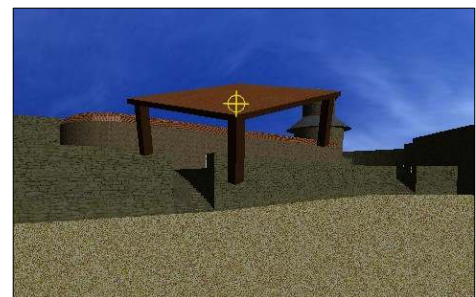
Les quelques lignes de code ci-contre déclarent la table comme `JiVe_BasicObject3D` (sans rapport avec les mathématiques), lui attribuent notamment un nom et une position de départ, ainsi que le fichier *Collada* associé. L'élément est ensuite ajouté à la scène pour obtenir la capture d'écran ci-contre.

```
// Table (basic object)
ArRef<JiVe_BasicObject3D> table = JiVe_BasicObject3D :: NEW(
  thisRef(),
  "/home/bastien/JiVe_prototype/models/table/table.dae",
  "table");
table -> setPosition(5200, 5200, 50);
table -> setCanMove(true);
addToScene(gameScene, table);
```



Placement

La dernière étape est le placement de l'élément au sein du monde virtuel. J'ai donc développé un outil permettant la manipulation des éléments grâce à un pointeur. Il suffit alors de trouver des coordonnées satisfaisantes, puis de remplacer celles visibles dans l'extrait de code illustrant la partie « Caractérisation », afin d'attribuer sa place à l'élément.



```
Attachment of the moving object to the pointer
Detachment of the moving object from the pointer
The object "table" is now on (5183.8 ; 5193.58 ; 154.715)
```

..... Déplacement dans un monde virtuel

Les modèles de déplacements

Afin de rendre une application 3D interactive, il faut permettre à l'utilisateur d'interagir avec le monde virtuel. En effet, La simple visualisation d'éléments 3D sous différents angles à l'aide d'un périphérique ne développe pas chez la personne un sentiment d'immersion. Toutefois, ce sentiment est nécessaire à son implication dans le jeu et à l'identification à son avatar.

Afin de développer cela, nous avons choisi de créer un monde fixe sur lequel reposent des éléments 3D manipulables, à l'instar du monde réel. L'utilisateur aura donc la possibilité de se déplacer et d'interagir avec cet environnement, lui donnant ainsi le statut d'acteur et non de spectateur.

Il a alors fallu choisir un mode de déplacement adapté aux besoins du projet. J'ai donc étudié différents modèles utilisés depuis la genèse des jeux vidéo 3D. Voici deux exemples de jeux dont j'ai retenu le système de déplacement :



« World of Warcraft »



« Quake III Arena »

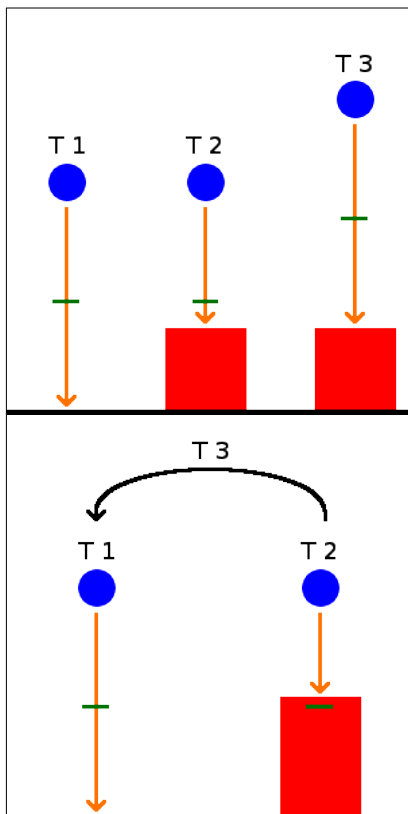
La première capture illustre le fait de placer la caméra à une distance de quelques mètres derrière un personnage 3D, appelé avatar, représentant le joueur. Ce système a comme avantages principaux de permettre au joueur de se représenter dans le jeu, ainsi que d'avoir une perception plus globale de son environnement. Ce mode de déplacement convient donc parfaitement aux besoins du projet concernant l'implication du joueur et la mise en ligne de l'application.

La seconde capture montre un système de déplacement propre aux FPS (First Person Shooter) : le joueur voit ce que son avatar est censé voir (caméra subjective). Contrairement au jeu précédent, ce mode de déplacement ne permet pas une bonne perception de l'environnement car l'angle de vue y est plus restreint. Il permet toutefois une manipulation plus intuitive du pointeur et en fait donc un système très intéressant pour JiVé Maths lors des activités nécessitant plus de précision.

Les déplacements dans le prototype

Le prototype aura pour but de présenter une activité représentant le projet JiVé Maths. J'ai donc développé en premier lieu le système de déplacement du second exemple, plus adapté à ce besoin. En effet, le premier n'est qu'une adaptation de celui-là, car il fonctionne de la même manière. Les seules différences sont l'ajout de l'avatar et le recul de la caméra.

Les déplacements sur un sol régulier fonctionnent ainsi : la caméra est placée à 1,70 mètres au dessus du sol et avance en restant à cette hauteur. Pour ce faire, on tire à chaque pas de temps un rayon invisible vers le sol qui nous renverra la distance avec le premier élément 3D croisé. Si cette distance n'est pas égale à la hauteur initiale, on la réajuste. Toutefois ce système doit être adapté aux objets rencontrés dans le monde virtuel. Les quelques schémas ci-dessous expliquent cela. La caméra y est représentée en bleu, les éléments du monde en rouge, et les rayons en orange.



Pour valider ou non un déplacement, il faut tout d'abord étudier si un objet est rencontré. En effet, tous n'ont pas la même hauteur, et un humain normal ne peut instantanément escalader un mur de cinq mètres de haut.

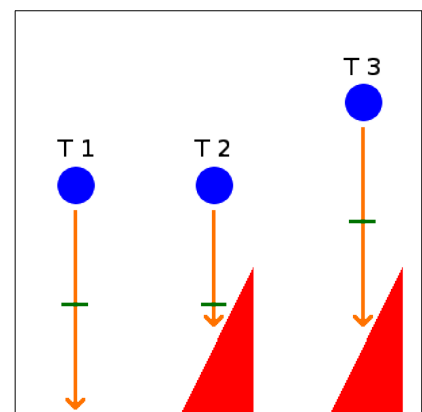
Une solution simple pour répondre à ce problème a été de définir un seuil de hauteur, représenté en vert sur les schémas ci-contre. La distance renvoyée par le rayon est comparée à cette valeur, et permet donc de déterminer si l'élément rencontré est franchissable ou non.

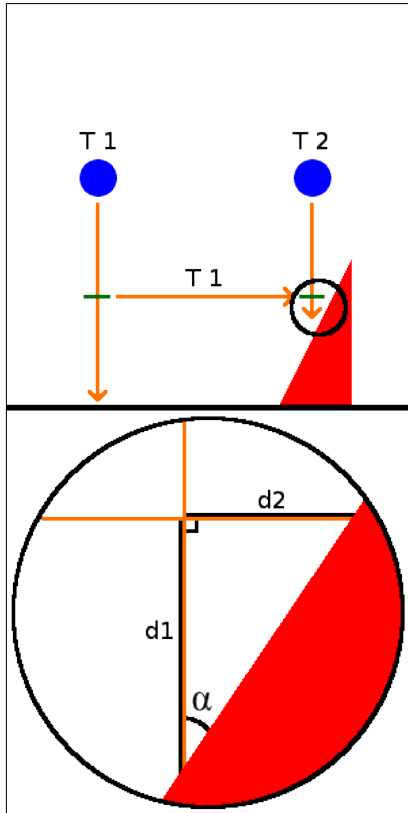
Le schéma du haut illustre la caméra se dirigeant vers un objet franchissable, et réajustant sa hauteur pour simuler l'ascension de celui-ci. Celui du bas montre un objet trop haut pour le seuil, en interdisant l'escalade.

Toutefois, ce système donnait l'impression d'avancer et de reculer de manière saccadée lors d'une collision avec un élément infranchissable. Ce problème a été réglé en effectuant une projection virtuelle de la caméra en T2, au pas de temps T1, validant ou non le futur déplacement.

Je me suis alors rendu compte d'un autre problème engendré par ce système de déplacement. En effet, il n'avait été testé qu'avec des objets globalement verticaux (cubes, escaliers...), et pas encore avec des pentes.

Le schéma ci-contre illustre le fait que les positions successives de la caméra, si elles étaient trop rapprochées, ne permettaient pas au seuil de hauteur de passer sous l'obstacle. Ainsi, la pente ci-contre était franchissable, bien que très accentuée.





Le problème aurait pu être contourné en augmentant le pas de temps entre deux positions de la caméra, mais des déplacements lents le causaient à nouveau.

Une pente étant caractérisée par son angle, j'ai décidé de le calculer, afin de définir un angle maximal pour une pente franchissable.

Pour ce faire, il a suffi de tirer à chaque pas de temps un second rayon depuis le seuil de hauteur franchissable, dans la direction du mouvement. A l'instar du premier, il renvoie une distance à l'élément 3D rencontré.

Par des calculs utilisant ces distances et les positions de la caméra en T1 et T2, j'ai pu déterminer le triangle rectangle ci-contre. Il a ensuite été simple d'obtenir le cosinus de l'angle α , et donc de se servir de la valeur obtenue comme seuil à vérifier.

Cette méthode permet à présent un système de déplacement fonctionnel et adapté à l'environnement virtuel.

..... Pointer et manipuler des éléments 3D

Afin de manipuler les éléments du monde virtuel, il est nécessaire d'avoir un pointeur affiché à l'écran. Celui-ci sert à désigner une zone précise de l'écran et éventuellement à désigner un objet à saisir. De plus, sa forme indique un grand nombre d'informations à l'utilisateur : le pointeur pourra devenir une clé quand une porte est visée ou une main quand le joueur pointe un objet manipulable.

Le moyen classique pour positionner un pointeur est l'utilisation de la souris. En effet, elle permet de parcourir deux axes, correspondant à la hauteur et à la largeur de l'écran, afin d'accéder potentiellement à chaque pixel. Toutefois, ce périphérique ne permet pas un accès à la profondeur de l'écran, qui est pourtant une dimension importante pour la 3D.

Pour me permettre cet accès, j'ai porté mon choix sur la *Wiimote* (manette de *Nintendo Wii*). Grâce à deux sources infra-rouges localisées près de l'écran, la *Wiimote* positionne deux points. En les reliant, on obtient un segment aux caractéristiques suivantes :



- Le milieu correspond à l'emplacement du pointeur visible par le joueur.
- L'inclinaison indique la position de la manette et permet de faire tourner le pointeur.
- La longueur permet de connaître la distance entre l'écran et la *Wiimote* et donc de positionner des éléments 3D dans le sens de la profondeur.

La capture ci-contre montre les points renvoyés par le capteur



de la *Wiimote* (en rouge et en vert), ainsi que le pointeur visible par le joueur (en jaune).

L'inclinaison de la manette y est visible, et l'écart entre les points rouges et verts, ici assez grand, correspond à un espacement d'environ un mètre entre l'écran et le périphérique (et donc la main du joueur).

Derrière le pointeur jaune, il est possible de distinguer une petite sphère : le pointeur 3D. Habituellement invisible pour le joueur, il permet de trouver quel élément 3D est réellement pointé. En effet, le pointeur 2D n'utilise que deux axes (largeur et hauteur de l'écran) et ne peut donc pas renvoyer d'informations concernant le monde virtuel.

Pour permettre cela, j'ai utilisé la même technique que celle utilisée pour la gestion des déplacements : en tirant un rayon depuis la caméra dans la direction indiquée par le pointeur 2D, on peut définir quel est le premier élément rencontré. Cela permet de nombreuses applications, comme déplacer des objets (exemple de la table en page 10).

..... Activité du prototype : « La balance »

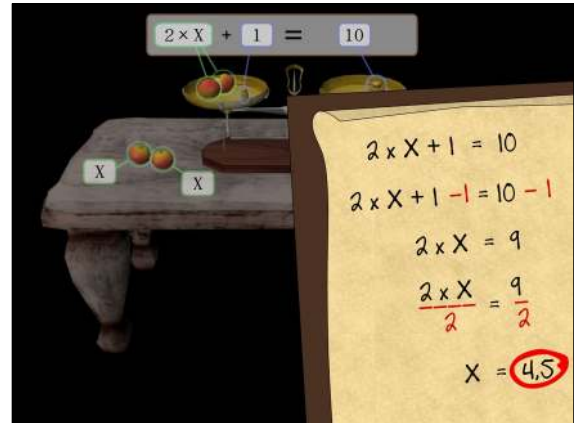
Une fois les outils de manipulation mis en place, j'ai pu commencer à programmer une des finalités du prototype : l'activité intitulée « La balance ». Les activités du projet étaient déjà définies à mon arrivée via des fiches descriptives :

La balance - Descriptif	
Utilisable dans divers endroits, en particulier dans le dépôt de chocolat	
Principe Une balance de Roberval magique reçoit sur ses plateaux divers objets pesants : - carrés, barres, et tablettes de chocolat de l'activité "Le dépôt de Chocolat" - des poids - 5g 10g 20g 50g 100g 200g 500g 1kg - autres (ex des onces (30,6 g)) - variables selon les problèmes, avec l'unité affichée sur la balance (g, 10g, 100g, kg, onces....) - des pommes, des poires et des... autres objets Les objets sont disponibles à côté de la balance et sont déplacés par le joueur, ou par une lévitation magique. Sur chaque plateau s'affiche l' expression algébrique correspondant à son chargement (ex : $5 + 3x$ si le plateau contient 5 onces et 3 pommes pesant x onces (nombre inconnu)) sous la balance s'affiche l'égalité ou l'inégalité correspondant à l'équilibre des plateaux ex : $5 + 3x = 7 + 2x$ Possibilité d'objets d' anti-matière (pour introduire des nombres négatifs dans les équations). - forme identique à l'objet, mais couleurs complémentaires (négatif couleur) - poids négatif : si on lâche l'objet, il "tombe" vers le haut - il se place sous le plateau de la balance, et pèse de son poids vers le haut - quand il rencontre son anti-objet, les deux s'annihilent dans un petit feu d'artifice Possibilité de refaire l'historique des manipulations. Les objets peuvent se grouper sur les plateaux selon des dispositions prévues. Modèle 3D à réaliser : La balance - Plateaux amovibles (identiques à ceux de l'activité du dépôt de chocolat) Animations : La balance a un mouvement naturel d'équilibre ou de déséquilibre en fonction du poids de chaque plateau. Si le poids est égal des deux cotés, la balance oscille doucement avant de se stabiliser à l'horizontale.	

L'élément 3D de la balance, représenté ci-contre, a été réalisé par une infographiste travaillant en collaboration avec l'entreprise.

Lors de la conception, elle a bien séparé chaque partie du modèle (plateaux, table, pommes...) de manière à nous permettre de les manipuler chacune séparément.

L'activité a pour but de résoudre une équation afin de déterminer le poids d'une pomme.



On peut imaginer le scénario de la manière suivante :

- Le joueur entre dans la salle et se trouve face à un personnage qui lui explique le problème à résoudre, à savoir déterminer le poids d'une pomme.
- Il choisit dans son inventaire le filtre « poids », permettant de mettre en évidence les propriétés correspondantes des éléments de la scène, puis les place sur les plateaux.
- Le programme étudie les propriétés des éléments en contact avec les plateaux de la balance, puis affiche l'équation correspondante.
- Le programme lance une animation faisant pencher ou équilibrer les plateaux de la balance pour montrer au joueur le résultat de son action.

Toutefois, l'activité n'est pour l'instant que théorique car la balance n'est pas encore fonctionnelle. Elle est actuellement en cours d'importation dans le monde virtuel car au moment de mon départ, j'étudiais la correspondance entre les parties du modèle 3D réalisé par l'infographiste et celles de l'objet manipulables dans le code. Une fois cette étape effectuée, il ne restera que l'animation des plateaux à programmer.



Conclusion

Etat actuel du projet

Arrivé à la fin de cette période au sein de l'entreprise JiVé Maths, le projet n'est bien évidemment pas terminé, étant donnée son ampleur. L'activité « La balance » est en cours de réalisation et sera continuée après mon départ pour arriver à un prototype globalement terminé, ne nécessitant que quelques améliorations ainsi que des tests par des utilisateurs externes au projet.

L'utilisation du logiciel *BOUML* pour développer l'application simplifie le travail collaboratif, ce qui permettra à mon encadrant, ainsi qu'aux futurs développeurs de continuer le projet. Toujours dans cette optique de continuation, j'ai pris soin de laisser derrière moi un code commenté, organisé et clair.

Bilan personnel

Pour ma part, je suis très content d'avoir passé ces deux mois et demi au sein de l'entreprise JiVé Maths. J'ai pu découvrir et manipuler de la 3D tout en avançant vers une réalisation concrète. J'ai aussi appris trois langages informatiques que je ne maîtrisais pas encore : *UML*, *Python* et *C++*. De plus, la confrontation à des problèmes de positionnement dans un monde virtuel m'a fait prendre conscience de l'utilisation pratique des mathématiques en programmation.

L'entreprise étant en cours de création, j'ai pu assister à son développement, à la recherche de financements, à la collaboration et négociation avec d'autres entreprises comme BookBéo et Artefacto. Impliqué, j'ai pu donner mon avis sur de nombreux points et aider à l'évolution de JiVé Maths.

Mes seuls regrets sont de ne pas avoir commencé la programmation 3D assez tôt à mon goût, et d'avoir laissé derrière moi du code parfois très améliorable, principalement par manque de temps.

Toutefois, je reste très satisfait de cette période car elle m'a beaucoup appris. Par la suite, je resterai en contact avec l'entreprise, humainement et professionnellement, car il est prévu que je retourne travailler sur le projet cet été.